

Relatório Técnico: Analisador Léxico (Micro C)

Análise de Implementação em Flex, Tracing de Casos de Teste e Fundamentação Teórica

Autores: Hugo Pagnozzi, Leonardo Loveira,
Maria Fernanda

Disciplina: Compiladores / Análise
Léxica

Arquivo Analisado:
microc.flex

1. Visão Geral e Estrutura do Flex

O analisador léxico foi desenvolvido utilizando a ferramenta **Flex** (Fast Lexical Analyzer Generator). O código-fonte em `microc.flex` está organizado nas seções padronizadas do Flex, delimitadas pelos divisores `%%`:

Estrutura / Elemento	Descrição e Funcionalidade no Código
<code>%{ ... %}</code>	Seção de Declarações C: Todo o código contido entre estes delimitadores é copiado literalmente para o início do arquivo gerado (<code>lex.yy.c</code>). Inclui cabeçalhos (<code>stdio.h</code> , <code>stdlib.h</code>), a definição da enumeração <code>TokenType</code> , a estrutura <code>YYSTYPE</code> , e variáveis de controle de posição (<code>linha_atual</code> , <code>coluna_atual</code>).
<code>%% ... %%</code>	Seção de Regras Léxicas: Delimita o bloco onde as Expressões Regulares (ER) e suas respectivas ações em C são mapeadas. O primeiro <code>%%</code> inicia as regras; o segundo encerra a seção.
<code>yylex()</code>	Função Principal do Scanner: Função gerada automaticamente pelo Flex que executa o Autômato Finito Determinístico (AFD). Retorna um inteiro representando o <code>TokenType</code> a cada chamada.
<code>yyin</code>	Ponteiro de Entrada (FILE*): Determina a origem do fluxo de caracteres. No método <code>main()</code> , é associado ao arquivo lido (<code>yyin = arquivo_fonte;</code>).
<code>yytext</code> e <code>yylen</code>	<code>yytext</code> contém a string exata (lexema) reconhecida pela regra atual. <code>yylen</code> indica o comprimento dessa string, sendo utilizado diretamente no cálculo do avanço de colunas (<code>coluna_atual += yylen</code>).
<code>yywrap()</code>	Tratamento de EOF: Chamada pelo Flex ao atingir o fim da entrada. Retornar <code>1</code> (como no código) sinaliza que não há mais arquivos para processar, encerrando a análise.

2. Decisões de Implementação

2.1. Controle de Lookahead e Ambiguidade de Operadores

O Flex gerencia o *lookahead* internamente buscando a **regra do maior casamento (Longest Match Rule)**. Isso resolve conflitos clássicos como a distinção entre atribuição (`=`) e igualdade (`==`). A regra `"=="` consome dois caracteres se possível; caso contrário, recua para `"="` . Da mesma forma, o operador `"-"` é diferenciado entre subtração simples e sinal de número negativo (ex: `-123`) através do padrão de número inteiro `"-"?[DIGIT]+` .

2.2. Gerenciamento de Buffer, Linhas e Colunas

O controle de posição é feito manualmente através das variáveis globais `linha_atual` , `coluna_atual` , `linha_erro` e `coluna_erro` :

- Ao encontrar o caractere , incrementa-se `linha_atual` e reseta-se `coluna_atual = 1` .
- Para todos os demais tokens e espaços (`[ ]+`), incrementa-se `coluna_atual += yylen` .
- Em caso de erro, a posição exata de início do erro é capturada (`coluna_erro = coluna_atual`) antes do avanço.

2.3. Descarte de Comentários (Desafio Cumprido)

Os comentários de linha são reconhecidos pela expressão regular `"//".*`. A ação associada é apenas atualizar o ponteiro de coluna (`coluna_atual += yyleng;`) sem executar comando `return`. Dessa forma, o scanner consome os caracteres até o fim da linha e prossegue imediatamente para a próxima captura, descartando o comentário sem gerar tokens.

2.4. Resposta ao Caso de Borda: Número seguido de Letra (ex: `123abc`)

O analisador trata `123abc` como um **Erro Léxico único** através da regra explícita `"-"?{DIGIT}+{LETRA}{ALFANUM}*`, exibindo a mensagem: *"Identificador nao pode começar com numero"*.

Justificativa da Decisão: Separar essa entrada em dois tokens (`TK_NUM, 123` e `TK_ID, abc`) mascararia um erro comum de digitação na fase léxica, postergando uma falha de sintaxe indesejada. Tratar como `TK_ERRO` imediato melhora a qualidade das mensagens de diagnóstico para o programador.

3. Validação nos Casos de Teste Obrigatórios

Caso 1: Entrada Principal (`soma = a + b * 2;`)

```
(TK_ID, soma) (TK_ASSIGN, =) (TK_ID, a) (TK_PLUS, +) (TK_ID, b) (TK_TIMES, *) (TK_NUM, 2)
(TK_SEMI, ;) (TK_EOF, )
```

Caso 2: Casos de Borda (String Vazia e Espaços/Tabulações)

Para arquivos vazios ou contendo apenas espaços/tabs, nenhum token é gerado além do marcador de encerramento:

```
(TK_EOF, )
```

Caso 3: Caracteres Não Reconhecidos (ex: `@` na linha 1, col 5)

```
ERRO LEXICO (linha 1, coluna 5): @
```

4. Questões Teóricas (Seção 7)

Questão 1: Por que o tratamento de palavras reservadas é feito depois de reconhecer o padrão de identificador?

Palavras reservadas (como `if`, `while`, `int`) seguem exatamente a mesma estrutura sintática/léxica dos identificadores genéricos (iniciadas por letra, seguidas de alfanuméricos: `{LETRA}{ALFANUM}*`). Se tentássemos criar regras ou autômatos separados antes, aumentaríamos significativamente a complexidade e o número de estados do AFD.

A abordagem ideal (e padrão em compiladores) é reconhecer a string primeiramente como um padrão genérico de identificador e, em seguida, efetuar uma busca em uma **Tabela de Símbolos / Tabela de Palavras Chave** (usando *hash table* ou busca binária). Se a string estiver na tabela, ela é convertida para o token da palavra reservada específica; caso contrário, é classificada como `TK_ID`.

Questão 2: O que é o token `TK_EOF` e por que ele é necessário?

O token `TK_EOF` (*End of File*) sinaliza explicitamente ao analisador sintático (Parser) que o fluxo de entrada do arquivo fonte chegou ao fim.

Ele é indispensável porque a gramática livre de contexto da linguagem precisa de um símbolo de encerramento (delimitador final) para validar se a estrutura do programa está completa e correta. Sem o `TK_EOF`, o parser não saberia se a entrada encerrou prematuramente no meio de um bloco estrutural incompleto ou se o processamento foi concluído com sucesso.

Questão 3: Para a entrada abaixo, qual é a sequência completa de tokens que um scanner correto deve gerar?

```
x = 10 // comentário  
y = 20;
```

Sequência Completa de Tokens Gerada:

```
(TK_ID, x) (TK_ASSIGN, =) (TK_NUM, 10) (TK_ID, y) (TK_ASSIGN, =) (TK_NUM, 20) (TK_SEMI, ;)  
(TK_EOF, )
```

Observação: O trecho `// comentário` e a quebra de linha subsequente são completamente filtrados e ignorados pelo scanner, garantindo que nenhum token correspondente ao comentário seja repassado às etapas subsequentes do compilador.